[Digital Voting System]

A digital voting system in Pakistan isn't just a leap into the future—it's a bold declaration that every vote will count, every voice will echo, and every citizen will shape the destiny of this nation, free from the shadows of fraud and inefficiency.

[Data Structure and Algorithm]
[Course Project]

FAZAIA BILQUIS COLLEGE OF EDUCATION FOR WOMEN

PAF NUR KHAN, RWP

Submitted by:

Sheema Zahra (22)

Javeria Kanwal (26)

Maheen Sajjad (77)

Submitted to:

MS Ayisha Bashir

Abstract

The Digital Voting System is an innovative software solution designed to modernize and streamline the electoral process, addressing the challenges associated with traditional voting methods. This project aims to create a secure, efficient, and user-friendly platform that facilitates voter registration, candidate management, and the voting process itself. By leveraging technology, the system enhances the accessibility and integrity of elections, ensuring that every eligible voter can participate in a transparent and straightforward manner.

The system allows voters to register by providing essential information such as their CNIC, name, father's name, date of birth, and gender. It incorporates robust authentication mechanisms to ensure that only eligible voters can cast their votes, thereby preventing fraudulent activities. Administrators have the capability to manage candidate information, including adding, updating, and removing candidates from the electoral process.

Once registered, voters can easily navigate the system to view a list of candidates and cast their votes. The system automatically tallies votes in real-time, providing immediate results and enhancing transparency in the electoral process. Additionally, the Digital Voting System includes search and sorting functionalities, allowing users to efficiently find specific voters or candidates.

Introduction

In the modern era, the need for efficient, secure, and user-friendly voting systems has become increasingly critical. Traditional voting methods often involve cumbersome paperwork, long queues, and the potential for human error or fraud. To address these challenges, we present the Digital Voting System, a comprehensive software solution designed to streamline the electoral process. This project aims to build a robust digital platform capable of managing and handling voter, candidate, and voting data in an efficient and effective manner.

Objectives of the Digital Voting System

- The primary objective of the Digital Voting System is to provide a seamless and paperless way for citizens to participate in elections. The system is designed with the following key goals in mind:
- **User -Friendly Interface:** The system offers an intuitive interface that allows voters to easily navigate through the voting process. This ensures that even individuals with minimal technical knowledge can participate without difficulty.

- **Efficient Data Management:** The system is capable of storing, updating, and retrieving voter and candidate information efficiently. This includes the ability to add new voters and candidates, as well as modify existing records as needed.
- **Secure Voting Process:** Security is a paramount concern in any electoral system. The Digital Voting System incorporates robust authentication mechanisms to ensure that only authorized users can access sensitive data. Admins have exclusive rights to manipulate voter and candidate data, thereby minimizing the risk of unauthorized access or tampering.
- **Real-Time Vote Counting:** The system allows for immediate tallying of votes, providing quick and accurate results. This feature enhances transparency and trust in the electoral process, as results can be generated and displayed in real-time.
- **Accessibility:** The Digital Voting System is designed to be accessible to all eligible voters, including those with disabilities. This inclusivity ensures that everyone has the opportunity to participate in the democratic process.

Key Features of the Digital Voting System

1. Voter Registration

The system allows for the easy registration of voters. Users can input their details, including their CNIC (Computerized National Identity Card), name, father's name, date of birth, and gender. The system validates this information to ensure that only eligible voters are registered.

2. Candidate Management

Admins can add new candidates to the system, including their candidate ID, name, party affiliation, and date of birth. This feature allows for the dynamic management of candidates throughout the electoral process.

3. Voting Process

Once registered, voters can cast their votes for their preferred candidates. The system provides a list of candidates, allowing voters to select their choice easily. Each voter can only cast one vote, ensuring the integrity of the election.

4. Admin Access

The system includes a dedicated admin interface that allows authorized personnel to manage voter and candidate data. Admins can add, update, or delete records as necessary, ensuring that the database remains current and accurate.

5. Search and Sort Functions

The Digital Voting System includes search functionalities that allow users to find specific voters or candidates by their CNIC or name. Additionally, the system can sort and display lists of voters and candidates, making it easier to manage large datasets.

6. Election Results Display

After the voting process is complete, the system can display the election results in a clear and organized manner. This feature includes the total number of votes received by each candidate, as well as the current leading candidate.

7. Data Persistence

The system ensures that all data is stored securely and can be retrieved as needed. Voter and candidate information, as well as voting results, are saved to files, allowing for easy access and management.

Functional Requirements

Functional requirements specify what the system should do. They describe the interactions between the system and its users, as well as the system's behavior in response to specific inputs.

User Registration

- The system shall allow eligible voters to register by providing their CNIC, name, father's name, date of birth, and gender.
- The system shall validate the CNIC to ensure it is 13 digits long and numeric.

Voter Authentication

- The system shall authenticate voters using their CNIC before allowing them to cast a vote.
- The system shall prevent users from voting more than once.

Candidate Management

- The system shall allow administrators to add new candidates by entering their candidate ID, name, party affiliation, and date of birth.
- The system shall allow administrators to update or remove existing candidates.

Voting Process

- The system shall display a list of candidates for voters to choose from.
- The system shall allow voters to cast their votes for a selected candidate.
- The system shall record the vote and update the candidate's vote count.

Vote Counting and Results Display

- The system shall automatically tally votes and display the results in real-time after the voting period ends.
- The system shall show the total number of votes received by each candidate and indicate the leading candidate.

Search Functionality

- The system shall allow users to search for voters and candidates by CNIC or name.
- The system shall provide sorting options for displaying voters and candidates based on different criteria (e.g., name, party).

Data Persistence

- The system shall save voter and candidate data to files to ensure data persistence across sessions.
- The system shall load existing data from files when it starts.

Admin Access

- The system shall provide a secure login for administrators to manage voter and candidate data.
- The system shall restrict access to sensitive functionalities to authorized personnel only.

Non-Functional Requirements

Non-functional requirements specify how the system performs its functions. They address the quality attributes of the system, such as performance, security, usability, and reliability.

Performance

- The system shall be able to handle at least 1,000 concurrent users without significant degradation in performance.
- The system shall respond to user actions (e.g., casting a vote, searching for candidates) within 2 seconds.

Security

- The system shall implement secure authentication mechanisms to protect voter data and prevent unauthorized access.
- The system shall encrypt sensitive data, such as CNICs and votes, to ensure confidentiality.
- The system shall log all administrative actions for auditing purposes.

Usability

- The system shall provide a user-friendly interface that is easy to navigate for both voters and administrators.
- The system shall include help documentation and tooltips to assist users in understanding how to use the system.

Reliability

- The system shall have an uptime of 99.9% during the voting period to ensure that voters can access it at all times.

- The system shall have backup mechanisms in place to prevent data loss in case of a failure.

Scalability

- The system shall be designed to accommodate an increasing number of users and data without requiring significant rework.
- The system shall allow for the addition of new features and functionalities in the future.

Maintainability

- The system shall be designed with modular components to facilitate easy updates and maintenance.
- The codebase shall follow best practices and coding standards to ensure readability and maintainability.

Accessibility

- The system shall comply with accessibility standards (e.g., WCAG) to ensure that it is usable by individuals with disabilities.
- The system shall provide alternative text for images and support keyboard navigation.

Code:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <list>
#include <string>
#include <regex>
#include <iomanip>
#include <vector>
#include <algorithm>
#include <unordered_map>
using namespace std;
// Data Structures
struct Voter {
    string cnic; // Add CNIC to the Voter struct
    string name;
    string father_name;
    string date_of_birth;
    string gender; // Add gender field
```

```
    bool voted = false; // Indicates if the voter has voted
};

struct Candidate {
    string candidate_id; // Add Candidate ID to the Candidate struct
    string name;
    string party;
    string date_of_birth;
    int votes = 0; // Initialize votes to 0
};

// Data Storage
list<Voter> voter_database;  // List of voters
list<Candidate> candidate_database;  // List of candidates
unordered_map<string, int> votes;  // Key: Candidate ID, Value: Vote Count

// Admin credentials
const string ADMIN_USERNAME = "admin";
const string ADMIN_PASSWORD = "admin123";

// Function Declarations
bool is_numeric(const string &str);
bool is_alphabetic(const string &str);
bool is_valid_date(const string &date);
string get_valid_cnic();
string get_valid_name(const string &prompt);
string get_valid_date();
void add_voter();
void update_voter();
void add_candidate();
void update_candidate();
void save_voters(const string &filename);
void save_candidates(const string &filename);
void load_voters(const string &filename);
```

```
void load_candidates(const string &filename);

void save_votes(const string &filename); // New function to save votes
void load_votes(const string &filename); // New function to load votes
bool authenticate_admin();
bool authenticate_voter(const string &cnic);
void cast_vote(const string &cnic, int candidate_index);

void display_all_voters();
void display_all_candidates();
void search_voter();
void search_candidate();
void search_voter_by_name();
void search_candidate_by_name();
void sort_and_display_voters();
void sort_and_display_candidates();
void display_results(); // Function to display election results

// Utility Functions
bool is_numeric(const string &str) {
    return all_of(str.begin(), str.end(), ::isdigit);
}

bool is_alphabetic(const string &str) {
    // Allow alphabetic characters and spaces
    return all_of(str.begin(), str.end(), [](char c) { return isalpha(c) || isspace(c); });
}

// Validate date in DD/MM/YYYY format
bool is_valid_date(const string &date) {
    regex date_pattern(R"(\d{2}/\d{2}/\d{4})"); // Match DD/MM/YYYY
    if (!regex_match(date, date_pattern)) {
        return false;
    }
```

```
    int day, month, year;
    sscanf(date.c_str(), "%d/%d/%d", &day, &month, &year);

    // Check if month is valid
    if (month < 1 || month > 12) {
        return false;
    }

    // Check if day is valid for the given month
    if (day < 1 || day > 31) {
        return false;
    }

    // Check for months with 30 days
    if ((month == 4 || month == 6 || month == 9 || month == 11) && day > 30) {
        return false;
    }

    // Check for February
    bool is_leap_year = (year % 4 == 0 && (year % 100 != 0 || year % 400 == 0));
    if (month == 2 && (is_leap_year ? day > 29 : day > 28)) {
        return false;
    }

    return true;
}

// Input Validation Wrappers
string get_valid_cnic() {
    string cnic;
    while (true) {
        cout << "Enter CNIC (13 digits): ";
        cin >> cnic;
```

```
        if (is_numeric(cnic) && cnic.length() == 13) {
            return cnic;
        } else {
            cout << "Invalid CNIC. It must be 13 digits. Try again.\n";
        }
    }
}

string get_valid_name(const string &prompt) {
    string name;
    while (true) {
        cout << prompt;
        getline(cin >> ws, name); // Use getline to read the full line and ignore leading whitespace
        if (!name.empty() && is_alphabetic(name)) {
            return name;
        } else {
            cout << "Invalid name. It must contain only alphabetic characters and spaces. Try again.\n";
        }
    }
}

string get_valid_date() {
    string date;
    while (true) {
        cout << "Enter Date of Birth (DD/MM/YYYY): ";
        cin >> date;
        if (is_valid_date(date)) {
            return date;
        } else {
            cout << "Invalid date. Please use the format DD/MM/YYYY. Try again.\n";
        }
    }
}
```

```
// Add Voter
void add_voter() {
    string cnic = get_valid_cnic();
    string name = get_valid_name("Enter Name: ");
    string father_name = get_valid_name("Enter Father's Name: ");
    string date_of_birth = get_valid_date();

    string gender;
    while (true) {
        cout << "Enter Gender (Male/Female/Other): ";
        cin >> gender;
        if (gender == "Male" || gender == "Female" || gender == "Other") {
            break; // Valid gender input
        } else {
            cout << "Invalid gender. Please enter Male, Female, or Other.\n";
        }
    }

    // Check if voter already exists
    for (const auto& voter : voter_database) {
        if (voter.cnic == cnic) {
            cout << "Voter with this CNIC already exists.\n";
            return;
        }
    }

    voter_database.push_back({cnic, name, father_name, date_of_birth, gender}); // Store voter
    cout << "Voter " << name << " added successfully.\n";
}
// update voters

void update_voter() {
    string cnic = get_valid_cnic();
    for (auto& voter : voter_database) {
```

```
if (voter.cnic == cnic) {
    cout << "Updating details for " << voter.name << ":\n";
    while (true) {
        cout << "What would you like to update?\n";
        cout << "1. Name\n";
        cout << "2. Father's Name\n";
        cout << "3. Date of Birth\n";
        cout << "4. Gender\n";
        cout << "5. Exit Update\n";
        cout << "Enter your choice: ";
        int choice;
        cin >> choice;

        switch (choice) {
            case 1: { // Update Name
                string name = get_valid_name("Enter new Name : ");
                if (!name.empty()) {
                    voter.name = name;
                }
                break;
            }
            case 2: { // Update Father's Name
                string father_name = get_valid_name("Enter new Father's Name : ");
                if (!father_name.empty()) {
                    voter.father_name = father_name;
                }
                break;
            }
            case 3: { // Update Date of Birth
                string date_of_birth;
                cout << "Enter Date of Birth (DD/MM/YYYY): ";
                cin >> date_of_birth;
                if (!date_of_birth.empty() && is_valid_date(date_of_birth)) {
                    voter.date_of_birth = date_of_birth;
```

```
                } else if (!date_of_birth.empty()) {
                    cout << "Invalid date. Please use the format DD/MM/YYYY.\n";
                }
                break;
            }
            case 4: { // Update Gender
                string gender;
                cout << "Enter new Gender (Male/Female/Other): ";
                cin >> gender;
                if (gender == "Male" || gender == "Female" || gender == "Other") {
                    voter.gender = gender;
                } else if (!gender.empty()) {
                    cout << "Invalid gender. Please enter Male, Female, or Other.\n";
                }
                break;
            }
            case 5: // Exit Update
                cout <<voter.name<<" data is updated\n"<< "Exiting update.\n";
                return;
            default:
                cout << "Invalid choice. Please try again.\n";
        }
    }
  }
}
cout << "Voter with this CNIC does not exist.\n";
}

// Add Candidate
void add_candidate() {
    string candidate_id = get_valid_cnic();  // Candidate ID validation is similar to CNIC
    string name = get_valid_name("Enter Candidate Name: ");
    string party = get_valid_name("Enter Party Name: ");
    string date_of_birth = get_valid_date();
```

```
    for (const auto& candidate : candidate_database) {
        if (candidate.candidate_id == candidate_id) {
            cout << "Candidate with this ID already exists.\n";
            return;
        }
    }

    candidate_database.push_back({candidate_id, name, party, date_of_birth}); // Store candidate
    cout << "Candidate " << name << " added successfully.\n";
}

// Update Candidate
void update_candidate() {
    string candidate_id = get_valid_cnic(); // Assuming candidate ID is validated as CNIC
    for (auto& candidate : candidate_database) {
        if (candidate.candidate_id == candidate_id) {
            cout << "Updating details for " << candidate.name << ":\n";
            while (true) {
                cout << "What would you like to update?\n";
                cout << "1. Name\n";
                cout << "2. Party\n";
                cout << "3. Date of Birth\n";
                cout << "4. Exit Update\n";
                cout << "Enter your choice: ";
                int choice;
                cin >> choice;

                switch (choice) {
                    case 1: { // Update Name
                        string name = get_valid_name("Enter new Candidate Name : ");
                        if (!name.empty()) {
                            candidate.name = name;
                        }
```

```
                break;
            }
            case 2: { // Update Party
                string party = get_valid_name("Enter new Party Name : ");
                if (!party.empty()) {
                    candidate.party = party;
                }
                break;
            }
            case 3: { // Update Date of Birth
                string date_of_birth;
                cout << "Enter Date of Birth (DD/MM/YYYY): ";
                cin >> date_of_birth;
                if (!date_of_birth.empty() && is_valid_date(date_of_birth)) {
                    candidate.date_of_birth = date_of_birth;
                } else if (!date_of_birth.empty()) {
                    cout << "Invalid date. Please use the format DD/MM/YYYY.\n";
                }
                break;
            }
            case 4: // Exit Update
               cout <<candidate.name<<" data is updated\n"<< "Exiting update.\n";
                return;
            default:
                cout << "Invalid choice. Please try again.\n";
        }
    }
  }
}
  cout << "Candidate with this ID does not exist.\n";
}
// Save Voter Data
void save_voters(const string &filename) {
  ofstream file(filename);
```

```
    if (!file) cerr << "Error opening file for writing: " << filename << endl;
        return;

    file << "CNIC,Name,Father's Name,Date of Birth,Gender\n"; // Header
    for (const auto &voter : voter_database) {
        file << voter.cnic << "," << voter.name << "," << voter.father_name << ","
            << voter.date_of_birth << "," << voter.gender << "\n"; // Save gender
    }
    file.close();
}

// Save Candidate Data
void save_candidates(const string &filename) {
    ofstream file(filename);
    if (!file) {
        cerr << "Error opening file for writing: " << filename << endl;
        return;
    }

    file << "Candidate ID,Name,Party,Date of Birth\n"; // Header
    for (const auto &candidate : candidate_database) {
        file << candidate.candidate_id << "," << candidate.name << "," << candidate.party << ","
            << candidate.date_of_birth << "\n";
    }
    file.close();
}

// Load Voter Data
void load_voters(const string &filename) {
    ifstream file(filename);
    string line;
    getline(file, line); // Skip header line

    while (getline(file, line)) {
```

```
        stringstream ss(line);
        string cnic, name, father_name, date_of_birth, gender;

        getline(ss, cnic, ',');
        getline(ss, name, ',');
        getline(ss, father_name, ',');
        getline(ss, date_of_birth, ',');
        getline(ss, gender, ','); // Load gender

        voter_database.push_back({cnic, name, father_name, date_of_birth, gender}); // Store voter
    }
    file.close();
}

// Load Candidate Data
void load_candidates(const string &filename) {
    ifstream file(filename);
    string line;
    getline(file, line); // Skip header line

    while (getline(file, line)) {
        stringstream ss(line);
        string candidate_id, name, party, date_of_birth;

        getline(ss, candidate_id, ',');
        getline(ss, name, ',');
        getline(ss, party, ',');
        getline(ss, date_of_birth, ',');

        candidate_database.push_back({candidate_id, name, party, date_of_birth}); // Store candidate
    }
    file.close();
}
```

```
// Implementation of save_votes function
void save_votes(const string &filename) {
    ofstream file(filename);
    if (!file) {
        cerr << "Error opening file for writing: " << filename << endl;
        return;
    }

    for (const auto &vote : votes) {
        file << vote.first << "," << vote.second << "\n"; // Save candidate ID and vote count
    }
    file.close();
}

// Implementation of load_votes function
void load_votes(const string &filename) {
    ifstream file(filename);
    if (!file) {
        cerr << "Error opening file for reading: " << filename << endl;
        return;
    }

    string line;
    while (getline(file, line)) {
        stringstream ss(line);
        string candidate_id;
        int vote_count;
        if (getline(ss, candidate_id, ',') && (ss >> vote_count)) {
            votes[candidate_id] = vote_count; // Load candidate ID and vote count
        }
    }
    file.close();
}
```

```
// Authenticate Admin
bool authenticate_admin() {
    string username, password;
    cout << "Enter Admin Username: ";
    cin >> username;
    cout << "Enter Admin Password: ";
    cin >> password;

    return (username == ADMIN_USERNAME && password == ADMIN_PASSWORD);
}

// Authenticate Voter
bool authenticate_voter(const string &cnic) {
    for (const auto& voter : voter_database) {
        if (voter.cnic == cnic) {
            return true; // Check if the CNIC exists in the voter database
        }
    }
    return false;
}

// Cast Vote
void cast_vote(const string &cnic, int candidate_index) {
    if (candidate_index < 1 || candidate_index > candidate_database.size()) {
        cout << "Invalid candidate selection.\n";
        return;
    }

    auto it = candidate_database.begin();
    advance(it, candidate_index - 1); // Move iterator to the selected candidate
    string candidate_id = it->candidate_id;

    for (auto& voter : voter_database) {
        if (voter.cnic == cnic) {
```

```
            if (voter.voted) {
                cout << "You have already voted. Your vote will not count again.\n"; // Message for
already voted
            } else {
                votes[candidate_id]++;
                voter.voted = true; // Mark the voter as having voted
                cout << "Vote cast successfully for " << it->name << ".\n";
            }
            return;
        }
    }
}

// Display All Voters
void display_all_voters() {
    cout << "\nVoter List:\n";
    cout << setw(15) << "CNIC" << setw(20)
         << "Name" << setw(20)
         << "Father's Name" << setw(15)
         << "DOB" << setw(10) << "Gender"
         << setw(10) << "Voted" << endl;
    for (const auto &voter : voter_database) {
        cout << setw(15) << voter.cnic
             << setw(20) << voter.name
             << setw(20) << voter.father_name
             << setw(15) << voter.date_of_birth
             << setw(10) << voter.gender
             << setw(10) << (voter.voted ? "Yes" : "No") << endl;
    }
}

// Display All Candidates
void display_all_candidates() {
    cout << "\nCandidate List:\n";
```

```
    cout << setw(5) << "No" << setw(20) << "Name" << setw(20) << "Party" << endl;
    int serial_no = 1;
    for (const auto &candidate : candidate_database) {
        cout << setw(5) << serial_no++
             << setw(20) << candidate.name
             << setw(20) << candidate.party << endl;
    }
}
// display result
void display_results() {
    cout << "\nCurrent Election Results:\n";
    cout << setw(5) << "No"
         << setw(20) << "Candidate Name" << setw(20) << "Party" << setw(10) << "Votes" << endl;
    int serial_no = 1;
    string leading_candidate;
    int max_votes = 0; // Initialize to 0

    for (const auto &candidate : candidate_database) {
        int vote_count = votes[candidate.candidate_id]; // Get the vote count for the candidate
        cout << setw(5) << serial_no++
             << setw(20) << candidate.name
             << setw(20) << candidate.party
             << setw(10) << vote_count << endl;

        // Determine the leading candidate only if they have votes
        if (vote_count > max_votes) {
            max_votes = vote_count;
            leading_candidate = candidate.name;
        }
    }

    if (max_votes > 0) {
        cout << "\nCurrent Leading Candidate: " << leading_candidate << " with " << max_votes << "
votes.\n";
```

```
    } else {
        cout << "\nNo votes have been cast yet.\n";
    }
}

// Search Voter by CNIC
void search_voter() {
    string cnic = get_valid_cnic();
    for (const auto& voter : voter_database) {
        if (voter.cnic == cnic) {
            cout << "Voter found: " << voter.name << ", Father's Name: " << voter.father_name
                << ", Date of Birth: " << voter.date_of_birth << ", Gender: " << voter.gender
                << ", Voted: " << (voter.voted ? "Yes" : "No") << endl;
            return;
        }
    }
    cout << "Voter with this CNIC does not exist.\n";
}

// Search Candidate by ID
void search_candidate() {
    string candidate_id = get_valid_cnic(); // Assuming candidate ID is also validated as CNIC
    for (const auto& candidate : candidate_database) {
        if (candidate.candidate_id == candidate_id) {
            cout << "Candidate found: " << candidate.name << ", Party: " << candidate.party << endl;
            return;
        }
    }
    cout << "Candidate with this ID does not exist.\n";
}

// Search Voter by Name
void search_voter_by_name() {
    string name;
```

```
    cout << "Enter Name to search: ";
    getline(cin >> ws, name);
    bool found = false;
    for (const auto &voter : voter_database) {
        if (voter.name == name) {
            cout << "Voter found: CNIC: " << voter.cnic << ", Name: " << voter.name
                << ", Father's Name: " << voter.father_name
                << ", Date of Birth: " << voter.date_of_birth
                << ", Gender: " << voter.gender
                << ", Voted: " << (voter.voted ? "Yes" : "No") << endl;
            found = true;
        }
    }
    if (!found) {
        cout << "No voters found with the name: " << name << endl;
    }
}

// Search Candidate by Name
void search_candidate_by_name() {
    string name;
 cout << "Enter Candidate Name to search: ";
    getline(cin >> ws, name);
    bool found = false;
    for (const auto &candidate : candidate_database) {
        if (candidate.name == name) {
            cout << "Candidate found: ID: " << candidate.candidate_id << ", Name: " << candidate.name
                << ", Party: " << candidate.party << endl;
            found = true;
        }
    }
    if (!found) {
        cout << "No candidates found with the name: " << name << endl;
    }
```

```
}

// Sort and Display Voters
void sort_and_display_voters() {
    vector<Voter> sorted_voters(voter_database.begin(), voter_database.end());
    sort(sorted_voters.begin(), sorted_voters.end(), [](const Voter &a, const Voter &b) {
        return a.name < b.name; // Sort by name
    }
    );

    cout << "\nSorted Voter List:\n";
    cout << setw(15) << "CNIC" << setw(20) << "Name" << setw(20) << "Father's Name"
         << setw(15) << "DOB" << setw(10) << "Gender" << setw(10) << "Voted" << endl;
    for (const auto &voter : sorted_voters) {
        cout << setw(15) << voter.cnic
             << setw(20) << voter.name
             << setw(20) << voter.father_name
             << setw(15) << voter.date_of_birth
             << setw(10) << voter.gender
             << setw(10) << (voter.voted ? "Yes" : "No") << endl;
    }
}

// Sort and Display Candidates
void sort_and_display_candidates() {
    vector<Candidate> sorted_candidates(candidate_database.begin(), candidate_database.end());
    sort(sorted_candidates.begin(), sorted_candidates.end(), [](const Candidate &a, const Candidate
&b) {
        return a.name < b.name; // Sort by name
    }
    );

    cout << "\nSorted Candidate List:\n";
    cout << setw(5) << "ID" << setw(20) << "Name" << setw(20) << "Party" << endl;
```

```
    for (const auto &candidate : sorted_candidates) {
        cout << setw(5) << candidate.candidate_id
             << setw(20) << candidate.name
             << setw(20) << candidate.party << endl;
    }
}

// Main Program
int main() {
    cout << "Welcome to the Secure Voting System\n";

    // Load data from files
    load_voters("voters.txt");
    load_candidates("candidates.txt");
    load_votes("votes.txt"); // Load votes when the program starts

    while (true) {
        cout << "\nAre you a Staff Member or a Voter?\n";
        cout << "1. Staff Member\n";
        cout << "2. Voter\n";
        cout << "3. Exit\n";
        cout << "Enter your choice: ";
        int role_choice;
        cin >> role_choice;

        if (role_choice == 1) {
            // Staff Member Menu
            if (authenticate_admin()) {
                while (true) {
                    cout << "\nStaff Member Menu:\n";
                    cout << "1. Add Voter\n";
                    cout << "2. Update Voter\n";
                    cout << "3. Add Candidate\n";
                    cout << "4. Update Candidate\n";
```

```
cout << "5. Display Voters\n";
cout << "6. Display Candidates\n";
cout << "7. Search Voter by CNIC\n";
cout << "8. Search Candidate by ID\n";
cout << "9. Search Voter by Name\n";
cout << "10. Search Candidate by Name\n";
cout << "11. Sort and Display Voters\n";
cout << "12. Sort and Display Candidates\n";
cout << "13. Save and Exit\n";
cout << "14. View Election Results\n"; // Corrected line
cout << "Enter your choice: ";
int staff_choice;
cin >> staff_choice;

if (staff_choice == 1) {
    add_voter();
} else if (staff_choice == 2) {
    update_voter();
} else if (staff_choice == 3) {
    add_candidate();
} else if (staff_choice == 4) {
    update_candidate();
} else if (staff_choice == 5) {
    display_all_voters();
} else if (staff_choice == 6) {
    display_all_candidates();
} else if (staff_choice == 7) {
    search_voter();
} else if (staff_choice ==  8) {
    search_candidate();
} else if (staff_choice == 9) {
    search_voter_by_name();
} else if (staff_choice == 10) {
    search_candidate_by_name();
```

```
            } else if (staff_choice == 11) {
                sort_and_display_voters();
            } else if (staff_choice == 12) {
                sort_and_display_candidates();
            } else if (staff_choice == 13) {
                save_voters("voters.txt");
                save_candidates("candidates.txt");
                save_votes("votes.txt"); // Save votes when exiting
                cout << "Data saved successfully. Exiting...\n";
                return 0;
            } else if (staff_choice == 14) { // Corrected line
                display_results(); // Display election results
            } else {
                cout << "Invalid choice. Please try again.\n";
            }
        }
    }
} else if (role_choice == 2) {
    // Voter Menu
    string cnic;
    cout << "Enter CNIC: ";
    cin >> cnic;
    if (authenticate_voter(cnic)) {
        while (true) {
            cout << "\nVoter Menu:\n";
            cout << "1. Display Candidates\n";
            cout << "2. Cast Vote\n";
            cout << "3. Exit\n";
            cout << "Enter your choice: ";
            int voter_choice;
            cin >> voter_choice;

            if (voter_choice == 1) {
                display_all_candidates();
```

```
            } else if (voter_choice == 2) {
                int candidate_index;
                cout << "Enter Serial Number to Vote For: ";
                cin >> candidate_index;
                cast_vote(cnic, candidate_index);
                break; // Exit voter menu after casting vote
            } else if (voter_choice == 3) {
                cout << "Exiting the system. Goodbye!\n";
                break;
            } else {
                cout << "Invalid choice. Please try again.\n";
            }
        }
    } else {
        cout << "Authentication failed. Please check your CNIC.\n";
    }
} else if (role_choice == 3) {
    cout << "Exiting the system. Goodbye!\n";
    break;
} else {
    cout << "Invalid choice. Please try again.\n";
}
}
return 0;
}
```

Output:

```
C:\Users\4STAR\Deskto

Welcome to the Secure Voting System

Are you a Staff Member or a Voter?
1. Staff Member
2. Voter
3. Exit
Enter your choice:
```

```
C:\Users\4STAR\Deskto

Welcome to the Secure Voting System

Are you a Staff Member or a Voter?
1. Staff Member
2. Voter
3. Exit
Enter your choice: 1
Enter Admin Username: admin
Enter Admin Password: admin123

Staff Member Menu:
1. Add Voter
2. Update Voter
3. Add Candidate
4. Update Candidate
5. Display Voters
6. Display Candidates
7. Search Voter by CNIC
8. Search Candidate by ID
9. Search Voter by Name
10. Search Candidate by Name
11. Sort and Display Voters
12. Sort and Display Candidates
13. Save and Exit
14. View Election Results
Enter your choice:
```

```
C:\Users\4STAR\Deskto

Staff Member Menu:
1. Add Voter
2. Update Voter
3. Add Candidate
4. Update Candidate
5. Display Voters
6. Display Candidates
7. Search Voter by CNIC
8. Search Candidate by ID
9. Search Voter by Name
10. Search Candidate by Name
11. Sort and Display Voters
12. Sort and Display Candidates
13. Save and Exit
14. View Election Results
Enter your choice: 1
Enter CNIC (13 digits): 1234567876543
Enter Name: Zara Noor
Enter Father's Name: Ahmed Raza
Enter Date of Birth (DD/MM/YYYY): 06/11/2000
Enter Gender (Male/Female/Other): Female
Voter Zara Noor  added successfully.
```

```
C:\Users\4STAR\Deskto

Staff Member Menu:
1. Add Voter
2. Update Voter
3. Add Candidate
4. Update Candidate
5. Display Voters
6. Display Candidates
7. Search Voter by CNIC
8. Search Candidate by ID
9. Search Voter by Name
10. Search Candidate by Name
11. Sort and Display Voters
12. Sort and Display Candidates
13. Save and Exit
14. View Election Results
Enter your choice: 3
Enter CNIC (13 digits): 1234567876532
Enter Candidate Name: Arham Ali
Enter Party Name: Independent
Enter Date of Birth (DD/MM/YYYY): 13/06/1997
Candidate Arham Ali  added successfully.
```

```
Enter your choice: 2
Enter CNIC (13 digits): 1234512345718
Updating details for  Noreen Malik:
What would you like to update?
1. Name
2. Father's Name
3. Date of Birth
4. Gender
5. Exit Update
Enter your choice: 2
Enter new Father's Name : Raza Malik
What would you like to update?
1. Name
2. Father's Name
3. Date of Birth
4. Gender
5. Exit Update
Enter your choice: 5
 Noreen Malikdata is updated
Exiting update.
```

```
Enter your choice: 4
Enter CNIC (13 digits): 1234567890130
Updating details for Khawaja Asif:
What would you like to update?
1. Name
2. Party
3. Date of Birth
4. Exit Update
Enter your choice: 2
Enter new Party Name : PLMN
What would you like to update?
1. Name
2. Party
3. Date of Birth
4. Exit Update
Enter your choice: 5
Invalid choice. Please try again.
What would you like to update?
1. Name
2. Party
3. Date of Birth
4. Exit Update
Enter your choice: 4
Khawaja Asifdata is updated
```

```
C:\Users\4STAR\Deskto

Staff Member Menu:
1. Add Voter
2. Update Voter
3. Add Candidate
4. Update Candidate
5. Display Voters
6. Display Candidates
7. Search Voter by CNIC
8. Search Candidate by ID
9. Search Voter by Name
10. Search Candidate by Name
11. Sort and Display Voters
12. Sort and Display Candidates
13. Save and Exit
14. View Election Results
Enter your choice: 5

Voter List:
           CNIC                Name       Father's Name            DOB    Gender     Voted
  1234512345671            Ali Khan       Muhammad Khan     01/01/1990      Male        No
  1234512345672         Fatima Bibi          Ahmed Bibi     02/02/1992    Female        No
  1234512345673           Usman Ali           Javed Ali     03/03/1985      Male        No
  1234512345674        Ayesha Malik         Saeed Malik     04/04/1995    Female        No
  1234512345675         Bilal Ahmed        Farooq Ahmed     05/05/1988      Male        No
  1234512345676           Sana Shah          Tariq Shah     06/06/1993    Female        No
  1234512345677         Hassan Raza            Raza Ali     07/07/1980      Male        No
  1234512345678         Zainab Noor       Noor Muhammad     08/08/1991    Female        No
  1234512345679         Omar Farooq        Farooq Ahmed     09/09/1987      Male        No
  1234512345680          Maria Khan          Abdul Khan     10/10/1994    Female        No
  1234512345681        Shahid Iqbal         Iqbal Ahmed     11/11/1982      Male        No
  1234512345682            Nida Ali            Ali Raza     12/12/1990    Female        No
  1234512345683         Kamran Shah         Shahid Khan     13/01/1988      Male        No
  1234512345684          Hina Malik         Malik Saeed     14/02/1995    Female        No
  1234512345685         Bilal Rafiq         Rafiq Ahmed     15/03/1986      Male        No
  1234512345686          Amina Bibi           Bibi Noor     16/04/1992    Female        No
  1234512345687           Saad Khan       Khan Muhammad     17/05/1989      Male        No
  1234512345688         Fatima Noor            Noor Ali     18/06/1993    Female        No
  1234512345689            Zain Ali            Ali Raza     19/07/1987      Male        No
  1234512345690          Sara Iqbal          Iqbal Raza     20/08/1991    Female        No
  1234512345691         Ahsan Malik         Malik Tariq     21/09/1985      Male        No
  1234512345692          Rabia Khan          Khan Saeed     22/10/1994    Female        No
  1234512345693         Waqas Ahmed          Ahmed Raza     23/11/1983      Male        No
  1234512345694          Sadaf Bibi           Bibi Noor     24/12/1990    Female        No
  1234512345695          Imran Raza            Raza Ali     25/01/1988      Male        No
  1234512345696         Ayesha Noor       Noor Muhammad     26/02/1992    Female        No
  1234512345697          Bilal Khan         Khan Farooq     27/03/1986      Male        No
  1234512345698          Hira Malik         Malik Saeed     28/04/1995    Female        No
```

1234512345753	Saad Iqbal	Iqbal Ahmed	21/11/1989	Male	No
1234512345754	Fatima Malik	Malik Tariq	22/12/1994	Female	No
1234512345755	Zainab Khan	Khan Farooq	23/01/1988	Male	No
1234512345756	Ahsan Raza	Raza Ali	24/02/1993	Female	No
1234512345757	Rabia Iqbal	Iqbal Raza	25/03/1986	Male	No
1234512345758	Waqas Noor	Noor Muhammad	26/04/1990	Female	No
1234512345759	Nida Rafiq	Rafiq Ahmed	27/05/1989	Male	No
1234512345760	Kamran Bibi	Bibi Noor	28/06/1992	Female	No
1234512345761	Aamir Ahmed	Ahmed Raza	29/07/1985	Male	No
1234512345762	Saira Malik	Malik Saeed	30/08/1990	Female	No
1234512345763	Farhan Raza	Raza Ali	31/09/1986	Male	No
1234512345764	Meher Khan	Khan Muhammad	01/10/1993	Female	No
1234512345765	Rizwan Iqbal	Iqbal Raza	02/11/1988	Male	No
1234512345766	Noreen Malik	Malik Tariq	03/12/1994	Female	No
1234512345767	Kamal Raza	Raza Ali	04/01/1986	Male	No
1234512345768	Amina Noor	Noor Muhammad	05/02/1991	Female	No
1234512345769	Bilal Iqbal	Iqbal Ahmed	06/03/1987	Male	No
1234512345770	Hira Rafiq	Rafiq Ahmed	07/04/1994	Female	No
1234512345771	Saif Malik	Malik Saeed	08/05/1989	Male	No
1234512345772	Fatima Raza	Raza Ali	09/06/1992	Female	No
1234512345773	Zainab Iqbal	Iqbal Ahmed	10/07/1986	Male	No
1234512345774	Ahsan Noor	Noor Muhammad	11/08/1994	Female	No
1234512345775	Rabia Khan	Khan Muhammad	12/09/1989	Male	No
1234512345776	Waqas Rafiq	Rafiq Ahmed	13/10/1993	Female	No
1234512345777	Nida Bibi	Bibi Noor	14/11/1985	Male	No
1234512345778	Kamran Malik	Malik Tariq	15/12/1990	Female	No
1234512345779	Aamir Raza	Raza Ali	16/01/1988	Male	No
1234512345780	Saira Noor	Noor Ali	17/02/1991	Female	No
1234512345781	Farhan Khan	Khan Farooq	18/03/1986	Male	No
1234512345782	Meher Iqbal	Iqbal Raza	19/04/1994	Female	No
1234512345783	Rizwan Malik	Malik Saeed	20/05/1989	Male	No
1234512345784	Noreen Raza	Raza Ali	21/06/1993	Female	No
1234512345785	Kamal Khan	Khan Muhammad	22/07/1985	Male	No
1234512345786	Amina Bibi	Bibi Noor	23/08/1990	Female	No
1234512345787	Bilal Raza	Raza Ali	24/09/1986	Male	No
1234512345788	Hina Noor	Noor Muhammad	25/10/1992	Female	No
1234512345789	Saad Iqbal	Iqbal Ahmed	26/11/1989	Male	No
1234512345790	Fatima Malik	Malik Tariq	27/12/1994	Female	No
1234512345791	Zainab Khan	Khan Farooq	28/01/1988	Male	No
1234512345792	Ahsan Raza	Raza Ali	29/02/1993	Female	No
1234512345793	Rabia Iqbal	Iqbal Raza	30/03/1986	Male	No
1234512345794	Waqas Noor	Noor Muhammad	31/04/1990	Female	No
1234512345795	Nida Rafiq	Rafiq Ahmed	01/05/1989	Male	No
1234512345796	Kamran Bibi	Bibi Noor	02/06/1992	Female	No
1234512345797	Aamir Ahmed	Ahmed Raza	03/07/1985	Male	No
1234512345798	Saira Malik	Malik Saeed	04/08/1990	Female	No
1234512345799	Farhan Raza	Raza Ali	05/09/1986	Male	No
1234512345800	Meher Khan	Khan Muhammad	06/10/1993	Female	No

```
Enter your choice: 6

Candidate List:
   No                Name                Party
    1      Shahbaz Sharif               PML-N
    2        Khawaja Asif                PLMN
    3      Sadiq Sanjrani                 BAP
    4       Shahid Afridi         Independent
    5          Rehana Dar         Independent
    6   Ali Amin Gandapur                 PTI
    7           Ali Wazir         Independent
    8          Imran Khan                 PTI
    9  Yusuf Raza Gillani                 PPP
   10   Hina Rabbani Khar                 PPP
   11Shahid Khaqan Abbasi               PML-N
   12Shahid Khaqan Abbasi               PML-N
   13 Raja Pervaiz Ashraf                 PPP
   14       Farooq Sattar              MQM-P
   15         Naseem Shah         Independent
   16     Fazal-ur-Rehman               JUI-F
   17         Zubair Umar               PML-N
   18      Omar Ayub Khan                 PTI
   19      Shehbaz Sharif               PML-N
   20              maheen                 PTI
   21        Nawaz Sharif               PML-N
   22  Dr. Shireen Mazari                 PTI
   23        Maryam Nawaz               PML-N
   24      Murad Ali Shah                 PPP
   25    Asif Ali Zardari                 PPP
   26      Gohar Ali Khan                 PTI
   27      Shireen Mazari                 PTI
   28Bilawal Bhutto Zardari                 PPP
```

```
Staff Member Menu:
1. Add Voter
2. Update Voter
3. Add Candidate
4. Update Candidate
5. Display Voters
6. Display Candidates
7. Search Voter by CNIC
8. Search Candidate by ID
9. Search Voter by Name
10. Search Candidate by Name
11. Sort and Display Voters
12. Sort and Display Candidates
13. Save and Exit
14. View Election Results
Enter your choice: 7
Enter CNIC (13 digits): 1234512345711
Voter found:  Waqas Raza, Father's Name:  Raza Ali, Date of Birth:  10/05/1988, Gender:  Male, Voted: No
```

```
Staff Member Menu:
1. Add Voter
2. Update Voter
3. Add Candidate
4. Update Candidate
5. Display Voters
6. Display Candidates
7. Search Voter by CNIC
8. Search Candidate by ID
9. Search Voter by Name
10. Search Candidate by Name
11. Sort and Display Voters
12. Sort and Display Candidates
13. Save and Exit
14. View Election Results
Enter your choice: 8
Enter CNIC (13 digits): 1234567890137
Candidate found: Hina Rabbani Khar, Party: PPP
```

```
Staff Member Menu:
1. Add Voter
2. Update Voter
3. Add Candidate
4. Update Candidate
5. Display Voters
6. Display Candidates
7. Search Voter by CNIC
8. Search Candidate by ID
9. Search Voter by Name
10. Search Candidate by Name
11. Sort and Display Voters
12. Sort and Display Candidates
13. Save and Exit
14. View Election Results
Enter your choice: 10
Enter Candidate Name to search: Farooq Sattar
Candidate found: ID: 1234567890149, Name: Farooq Sattar, Party: MQM-P
```

```
Staff Member Menu:
1. Add Voter
2. Update Voter
3. Add Candidate
4. Update Candidate
5. Display Voters
6. Display Candidates
7. Search Voter by CNIC
8. Search Candidate by ID
9. Search Voter by Name
10. Search Candidate by Name
11. Sort and Display Voters
12. Sort and Display Candidates
13. Save and Exit
14. View Election Results
Enter your choice: 13
Data saved successfully. Exiting...

--------------------------------
Process exited after 146.9 seconds with return value 0
Press any key to continue . . .
```

```
C:\Users\4STAR\Deskto

Welcome to the Secure Voting System

Are you a Staff Member or a Voter?
1. Staff Member
2. Voter
3. Exit
Enter your choice: 2
Enter CNIC: 1234512345790

Voter Menu:
1. Display Candidates
2. Cast Vote
3. Exit
Enter your choice: 1

Candidate List:
   No                Name                Party
    1      Shahbaz Sharif               PML-N
    2        Khawaja Asif               PML-N
    3      Sadiq Sanjrani                 BAP
    4       Shahid Afridi         Independent
    5          Rehana Dar         Independent
    6   Ali Amin Gandapur                 PTI
    7           Ali Wazir         Independent
    8          Imran Khan                 PTI
    9  Yusuf Raza Gillani                 PPP
   10   Hina Rabbani Khar                 PPP
   11Shahid Khaqan Abbasi               PML-N
   12Shahid Khaqan Abbasi               PML-N
   13 Raja Pervaiz Ashraf                 PPP
   14       Farooq Sattar               MQM-P
   15         Naseem Shah         Independent
   16     Fazal-ur-Rehman               JUI-F
   17         Zubair Umar               PML-N
   18      Omar Ayub Khan                 PTI
   19      Shehbaz Sharif               PML-N
   20              maheen                 PTI
   21        Nawaz Sharif               PML-N
   22  Dr. Shireen Mazari                 PTI
   23        Maryam Nawaz               PML-N
   24      Murad Ali Shah                 PPP
   25    Asif Ali Zardari                 PPP
   26      Gohar Ali Khan                 PTI
   27      Shireen Mazari                 PTI
   28Bilawal Bhutto Zardari                 PPP
```

```
Voter Menu:
1. Display Candidates
2. Cast Vote
3. Exit
Enter your choice: 2
Enter Serial Number to Vote For: 3
Vote cast successfully for Sadiq Sanjrani.

Are you a Staff Member or a Voter?
1. Staff Member
2. Voter
3. Exit
Enter your choice: 2
Enter CNIC: 1234512345790

Voter Menu:
1. Display Candidates
2. Cast Vote
3. Exit
Enter your choice: 2
Enter Serial Number to Vote For: 2
You have already voted. Your vote will not count again.
```

```
Enter your choice: 14

Current Election Results:
   No      Candidate Name              Party     Votes
    1      Shahbaz Sharif              PML-N         0
    2        Khawaja Asif              PML-N         3
    3      Sadiq Sanjrani                BAP         1
    4       Shahid Afridi         Independent         0
    5          Rehana Dar         Independent         0
    6   Ali Amin Gandapur                PTI         0
    7           Ali Wazir         Independent         0
    8          Imran Khan                PTI         0
    9  Yusuf Raza Gillani                PPP         0
   10   Hina Rabbani Khar                PPP         0
   11Shahid Khaqan Abbasi              PML-N         0
   12Shahid Khaqan Abbasi              PML-N         0
   13 Raja Pervaiz Ashraf                PPP         0
   14       Farooq Sattar              MQM-P         0
   15         Naseem Shah         Independent         0
   16     Fazal-ur-Rehman               JUI-F         0
   17         Zubair Umar              PML-N         0
   18      Omar Ayub Khan                PTI         0
   19      Shehbaz Sharif              PML-N         0
   20              maheen                PTI         0
   21        Nawaz Sharif              PML-N         0
   22  Dr. Shireen Mazari                PTI         0
   23        Maryam Nawaz              PML-N         0
   24      Murad Ali Shah                PPP         0
   25    Asif Ali Zardari                PPP         0
   26      Gohar Ali Khan                PTI         0
   27      Shireen Mazari                PTI         0
   28Bilawal Bhutto Zardari                PPP         0

Current Leading Candidate: Khawaja Asif with 3 votes.
```